Despliegue y Validación en Entorno Local de una Arquitectura de Microservicios para Comercio Electrónico con Mediador Centralizado y Persistencia Aislada

Mark Osorio
Facultad de Ingeniería
Universidad Mesoamericana (UMES)
Quetzaltenango, Guatemala
osoriomark@umes.edu.gt

Cátedra: Arquitectura de Sistemas I
Ingeniería en Sistemas, Informática y Ciencias de la Computación
Universidad Mesoamericana
Ciclo 2026 – Proyecto Final (Valor: 15 pts)

***Resumen*—** Este informe de avance técnico documenta el diseño, la puesta en marcha local y la verificación funcional de un ecosistema distribuido de comercio electrónico orientado a microservicios. La arquitectura implementada se compone de tres proyectos de backend independientes: un servicio mediador (Broker) que funge como punto único de entrada y gestor de autenticación basado en tokens JWT, un microservicio de Productos desarrollado bajo el patrón Modelo-Vista-Controlador (MVC), y un microservicio de Pedidos estructurado en arquitectura por capas. La persistencia se diseñó mediante bases de datos aisladas en MySQL 8.4 gestionadas por medio de TypeORM. La orquestación completa del entorno se resolvió mediante Docker Compose, garantizando aislamiento de redes y control estricto de exposición de puertos. Las pruebas de integración se validaron con Postman contra la dirección local del broker, verificando tanto el rechazo sistemático de peticiones anónimas como el flujo transaccional de verificación y descuento de existencias.

***Términos de indexación*—** Arquitectura de Microservicios, API Gateway, Broker, Autenticación JWT, Docker Compose, Modelo-Vista-Controlador (MVC), Arquitectura por Capas, TypeORM, Postman.

I. INTRODUCCIÓN

El diseño de sistemas empresariales modernos exige desacoplar las responsabilidades de negocio para garantizar escalabilidad, tolerancia a fallos e independencia en el ciclo de vida del software. Frente a los modelos monolíticos tradicionales, las arquitecturas orientadas a servicios distribuidos permiten aislar dominios funcionales críticos, como la gestión de catálogos y el procesamiento transaccional de órdenes de compra.

En el marco del proyecto final de la cátedra de Arquitectura de Sistemas I de la Universidad Mesoamericana, se estableció como requisito la construcción individual de una plataforma de comercio electrónico compuesta por tres componentes de backend desacoplados, sin interfaz gráfica de usuario, demostrable exclusivamente mediante herramientas de prueba de API (Postman) y orquestada en contenedores.

El presente informe detalla la fase inicial del proyecto, enfocada en la definición de los límites arquitectónicos de cada servicio, la configuración de la persistencia aislada con mapeo objeto-relacional (ORM), la orquestación integral en entorno local con Docker y Docker Compose, y la verificación empírica de las reglas de autenticación e integración interservicios.

II. DISEÑO ARQUITECTÓNICO DEL ECOSISTEMA

El sistema se organiza bajo el patrón de diseño de puerta de enlace o mediador (*Broker/Mediator Pattern*), en el cual ningún servicio de negocio se expone directamente a los clientes externos. Toda solicitud proveniente del exterior debe atravesar el broker, que asume la responsabilidad de autenticación previa a cualquier redirección.

A. Microservicio Broker / Mediador (Puerto 3000)

El Broker actúa como fachada perimetral y punto de entrada único (*Single Point of Entry*) al sistema. Se implementó utilizando NestJS sobre Node.js y TypeScript. Su comportamiento se apega rigurosamente a las siguientes directrices:

- **Cero lógica de negocio:** No procesa reglas comerciales, cálculos de inventario ni transacciones financieras.
- **Autenticación centralizada:** Valida la presencia e integridad de tokens JWT transmitidos en la cabecera HTTP `Authorization: Bearer <token>` mediante la biblioteca criptográfica `jose`.
- **Enrutamiento transparente:** Redirige las solicitudes validadas hacia los servicios internos (`http://productos:3001` y `http://pedidos:3002`) a través de un proxy HTTP interno, inyectando un token de firma de servicio (`SERVICIO_TOKEN`) para autenticar la comunicación interna.
- **Manejo de acceso:** Incorpora un subsistema de credenciales para usuarios administradores y clientes, permitiendo revocar accesos y denegar operaciones a identidades suspendidas.

B. Microservicio de Productos (Puerto 3001)

Encargado exclusivo de la administración del catálogo y el control de existencias. Su diseño sigue el patrón arquitectónico **Modelo-Vista-Controlador (MVC)**:

- **Modelo:** Entidad `Producto` mapeada con TypeORM hacia la tabla correspondiente, almacenando identificador autoincremental, nombre, precio unitario y existencias (*stock*).
- **Vista:** Clases de transferencia de datos (DTOs) y transformadores REST (`ProductoVista`, `DescuentoVista`) que moldean las respuestas JSON hacia el cliente, asegurando la serialización y ocultamiento de atributos internos.
- **Controlador:** Expone endpoints para el CRUD completo de productos y validaciones numéricas estrictas (precio > 0, stock ≥ 0). Asimismo, implementa la ruta crítica de descuento de existencias requerida por el servicio de pedidos.

C. Microservicio de Pedidos (Puerto 3002)

Responsable del registro y control de ciclo de vida de las compras. Está construido bajo una **Arquitectura en Capas** que desacopla la recepción HTTP de la lógica de aplicación y el acceso a datos:

- **Capa de Presentación:** Controladores que capturan las peticiones de creación y consulta de pedidos, validando los parámetros de entrada mediante `class-validator`.
- **Capa de Negocio / Dominio:** Orquesta la lógica del pedido. Antes de asentar la orden, invoca al cliente de productos a través del Broker para verificar la existencia del artículo y la disponibilidad del stock solicitado.
- **Capa de Persistencia / Datos:** Repositorio TypeORM que almacena los registros con identificador UUIDv4, estado de la orden (`PENDIENTE`, `CONFIRMADO`, `RECHAZADO`) y fecha de creación.
- **Idempotencia y Manejo de Fallos:** Si la consulta de inventario o el descuento falla (por existencia insuficiente o tiempo de espera agotado), la orden se transiciona al estado correspondiente sin inconsistencias en el inventario.

D. Persistencia y Aislamiento de Datos

Para cumplir con el principio de persistencia políglota y bases de datos independientes por microservicio, se configuró una instancia de MySQL 8.4 que aloja tres esquemas completamente separados:

- `broker`: Almacena usuarios, credenciales hasheadas y estado de autenticación.
- `productos`: Almacena catálogo, descripciones, precios e inventario físico.
- `pedidos`: Almacena órdenes de compra, referencias cruzadas y estados de procesamiento.

Cada base de datos cuenta con un usuario SQL independiente con privilegios acotados únicamente a su esquema, imposibilitando la lectura cruzada de tablas a nivel de base de datos.

III. ORQUESTACIÓN Y PUESTA EN MARCHA CON DOCKER

La totalidad de los servicios se empaquetó en contenedores Docker y se orquestó mediante un archivo `compose.yaml`.

A. Aislamiento de Redes y Control de Puertos

En una arquitectura de microservicios robusta, los servicios de dominio no deben ser alcanzables directamente desde redes públicas. Por ello, la directiva de puertos de Docker Compose se configuró de la siguiente forma:

- **Broker:** Publica el puerto `127.0.0.1:3000:3000`. Es la única compuerta accesible desde la máquina anfitriona y herramientas externas.
- **Productos:** Escucha en el puerto `3001` únicamente dentro de la red privada de Docker (`tienda`). No expone puertos al host.
- **Pedidos:** Escucha en el puerto `3002` dentro de la red privada de Docker (`tienda`). No expone puertos al host.
- **MySQL:** Expone el puerto `3306` para tareas de mantenimiento local, implementando una directiva de *healthcheck* con `mysqladmin ping` para asegurar que los servicios dependientes arranquen únicamente tras la disponibilidad del motor SQL.

B. Evidencias de Ejecución

La Fig. 1 muestra la salida de la consola durante la recreación y el levantamiento ordenado de los tres microservicios en segundo plano, así como el listado de estado generado con el comando `docker compose ps`. Se evidencia que los cuatro contenedores (`tienda-mysql-1`, `tienda-broker-1`, `tienda-pedidos-1` y `tienda-productos-1`) se encuentran en ejecución activa (*healthy* y *Up*).

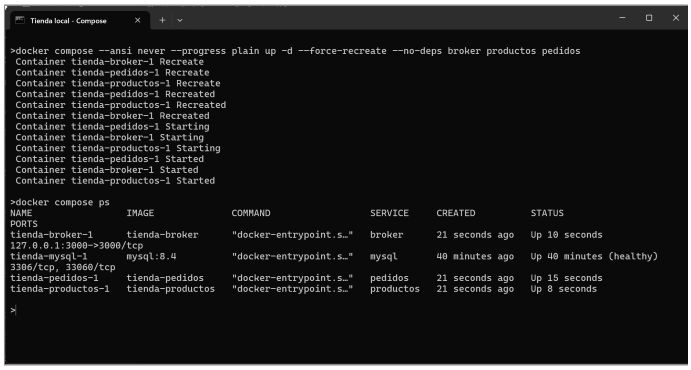


Fig. 1. Consola de comandos evidenciando la recreación de contenedores y la verificación de estados con `docker compose ps`.

Por su parte, la Fig. 2 presenta el panel gráfico de Docker Desktop dentro de la sección *Containers*. Se aprecia el grupo `tienda` expandido, con los cuatro contenedores del ecosistema operando con indicadores de estado activos (verde), consumo

de recursos balanceado y la correcta vinculación del puerto `3000:3000` asignado al contenedor del broker.

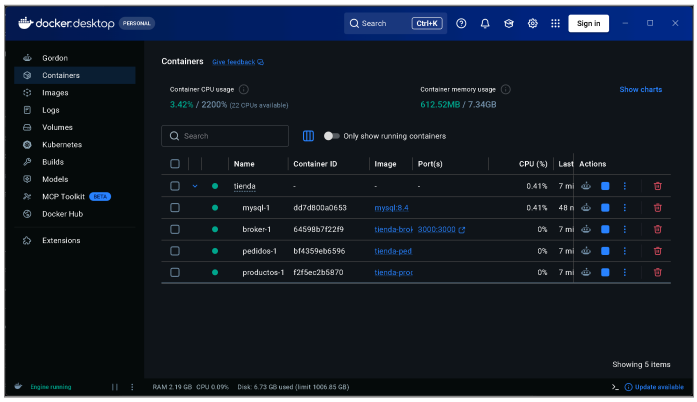


Fig. 2. Panel de Docker Desktop con el grupo de contenedores `tienda` expandido y en ejecución normal.

IV. PRUEBAS FUNCIONALES Y VALIDACIÓN EN LOCAL

A. Validación de Seguridad y Rechazo Sin Token

Para verificar el cumplimiento del criterio de seguridad del mediador, se realizaron pruebas contra la ruta protegida `http://localhost:3000/productos` prescindiendo de cabeceras de autorización.

Como se documenta en la Fig. 3, el Broker interceptó inmediatamente la petición y emitió una respuesta con código de estado HTTP **401 Unauthorized**, retornando el cuerpo JSON estructurado:

```
{
  "message": "Token ausente, inválido o expirado",
  "error": "Unauthorized",
  "statusCode": 401
}
```

Esto corrobora empíricamente que ninguna solicitud sin credenciales válidas logra penetrar hacia el microservicio interno de productos.

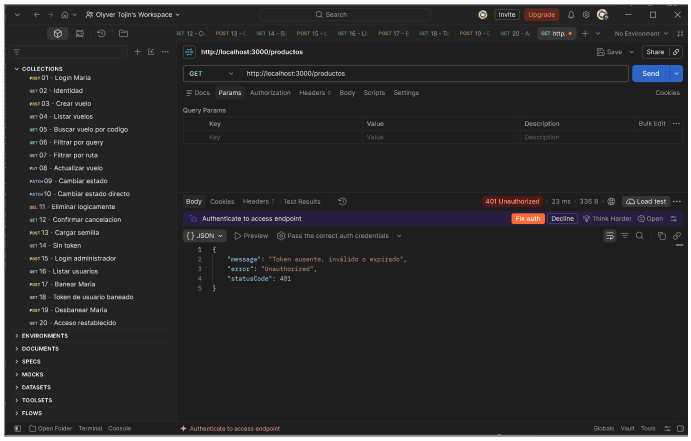


Fig. 3. Prueba en Postman dirigida a `http://localhost:3000/productos` sin token de autorización, demostrando el rechazo 401 Unauthorized.

B. Pruebas Unitarias Mediante Mocks

En cumplimiento con los requerimientos funcionales de la cátedra, se implementaron suites de pruebas unitarias automatizadas basadas en dobles de prueba (*mocks*) utilizando el módulo nativo `node:test` y las utilidades `t.mock.fn` de Node.js:

- **Confirmación de Pedido:** Se simuló la respuesta afirmativa del microservicio de productos (stock = 5), verificando que la orden transiciona a estado `CONFIRMADO` y descuenta la cantidad exacta.
- **Rechazo por Stock Insuficiente:** Se mockeó una disponibilidad menor a la demanda, comprobando que la capa de dominio dispara una excepción `ConflictException` y cancela la llamada a descuento.
- **Tolerancia a Timeout:** Se simuló una caída o demora del microservicio de catálogo lanzando `GatewayTimeoutException`, validando que el pedido permanece en estado `PENDIENTE` para garantizar idempotencia en reintentos.

C. Matriz de Endpoints y Cobertura Postman

Se estructuró una colección de Postman con 20 escenarios de prueba automatizados mediante Newman, evaluando generación de JWT tras autenticación en `POST /auth/login`, inserción y lectura de catálogo mediante `POST /productos` y `GET /productos`, flujo transaccional de compra en `POST /pedidos`, validación de existencias posteriores al descuento y manejo de casos de borde (precios negativos con 400, producto inexistente con 404 y stock agotado con 409).

V. ESTADO DE AVANCE FRENTE A LA RÚBRICA

La Tabla I resume la correspondencia entre los criterios de evaluación estipulados en el enunciado del proyecto final y los componentes consolidados en este informe de avance.

TABLA I
MATRIZ DE CUMPLIMIENTO DE LA RÚBRICA DE EVALUACIÓN

Criterio Evaluado	Puntos	Estado en el Presente Avance
Arquitectura general: tres servicios independientes con su propia arquitectura.	4 pts	**Completado:** Broker (Mediator), Productos (MVC) y Pedidos (Capas).
Autenticación por token en Broker: rechaza peticiones sin token válido.	2 pts	**Completado:** Verificado en Postman con HTTP 401 y JWT de sesión.
Comunicación correcta entre servicios.	3 pts	**Completado:** Pedidos invoca a Productos pasando obligatoriamente por Broker.
Pruebas unitarias con mocks.	2 pts	**Completado:** Cobertura de casos exitosos, conflicto de stock y timeout.
Documentación por Swagger.	1 pt	**Implementado:** Módulos `@nestjs/swagger` configurados por servicio.
Publicación en la nube (VPS).	2 pts	**Fase siguiente:** En preparación para el despliegue final.
Demostración individual funcional.	1 pt	**Fase final:** Preparada con colección de Postman (16 peticiones / 19 aserciones).
Total Ponderación del Curso	**15 pts**	**Avance consolidado conforme a especificaciones.**

VI. CONCLUSIONES Y PRÓXIMOS PASOS

La puesta en marcha local ha validado satisfactoriamente los pilares de desacoplamiento, seguridad y contención de datos requeridos para el proyecto final de Arquitectura de Sistemas I. El uso de Docker Compose ha simplificado el ciclo de vida de los servicios y ha garantizado que el microservicio Broker cumpla rigurosamente su función como compuerta única de entrada.

Los próximos pasos contemplan el aprovisionamiento y despliegue del ecosistema en el servidor virtual en la nube (VPS), la configuración de nombres de dominio y certificados SSL/TLS para el acceso seguro al broker público, y la ejecución de la suite completa de pruebas Newman para la entrega y demostración individual ante el docente.

REFERENCIAS

[1] M. Fowler, "Microservices: a definition of this new architectural term," *martinfowler.com*, 2014.

[2] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.

[3] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.

[4] IEEE, "ISO/IEC/IEEE Systems and software engineering -- Architecture description," *ISO/IEC/IEEE 42010:2011*, 2011.